

Starter Queries — Build Block

Copy these into `explore.py` and run them. Change the SQL to explore further.

How to use these: Open `explore.py` in the repo root. Replace the SQL inside the triple quotes with the query below. Save. Run:

Windows: `python explore.py` **Mac/Linux:** `python3 explore.py`

Getting column names: Change `.fetchall()` to `.fetchdf()` in your `explore.py` and print with `print(df.to_string())`. This gives you a labeled table instead of bare tuples — much easier to read.

Each query has a *"Try changing..."* prompt. That's where the discovery happens — the Gotcha you need for your reference doc shows up when you modify the query and the number moves (or doesn't) in a way you didn't expect.

Reading SQL — just enough for today:

- SELECT = which columns to show
- FROM = which table to look in
- WHERE = a filter — only show rows matching this condition
- GROUP BY = summarize: show one row per category instead of one row per record
- ORDER BY = sort the results
- count(*) = "how many rows match?"
- avg(...) = "what's the average?"
- round(...) = round to a whole number
- JOIN ... ON = combine two tables by matching a shared column
- LIMIT 5 = "just show me the first 5 rows"

You don't need to write SQL from scratch — copy the queries below, modify what the *"Try changing..."* prompt suggests, and read the output.

Attrition

Tables: `fct_separation`, `rpt_attrition_monthly`, `dim_worker_snapshot`

Query 1 — Look at the raw data

```
SELECT * FROM fct_separation LIMIT 5
```

Notice which columns exist. What types of workers are in here?

Here's what you'll see (using `.fetchdf()`):

	worker_id	employee_number	separation_date	separation_type	is_regrettable	org_unit_v2	worker_type
	separation_id						
0	W100003	E100003	2026-03-25	Involuntary	False	OU-1100	Regular
	SP000001						
1	W100006	E100006	2026-03-18	Voluntary	False	OU-2100	Regular
	SP000002						
2	W100009	E100009	2024-04-28	Voluntary	False	OU-4000	Regular
	SP000003						
3	W100017	E100017	2025-06-24	Voluntary	False	OU-4200	Regular
	SP000004						
4	W100026	E100026	2025-02-07	Voluntary	False	OU-4100	Regular
	SP000005						

Reading the columns: worker_id / employee_number = who left. separation_type = Voluntary (686) or Involuntary (240). is_regrettable = the org's judgment — true (231) or false (695). worker_type = Regular (717), Contingent (177), Intern (32) — contractors are in here too.

Key thing: This table has the numerator (separations) but *no denominator* (headcount). You can't compute a rate from this table alone.

Query 2 — Who left in 2025?

```
SELECT worker_type, count(*) AS n
FROM fct_separation
WHERE year(separation_date) = 2025
GROUP BY worker_type
ORDER BY 1
```

Try changing: remove the worker_type filter entirely. How many total separations? Should all of them be in your attrition rate?

Query 3 — Count the Regular separations

```
SELECT count(*) AS separations_2025
FROM fct_separation
WHERE year(separation_date) = 2025
AND worker_type = 'Regular'
```

This is your numerator. Now you need a denominator — but from where?

Query 4 — Check the other attrition table

```
SELECT * FROM rpt_attrition_monthly
ORDER BY month DESC LIMIT 5
```

How many columns does this table have? Could you compute a rate from it alone?

Query 5 — Get a denominator (beginning of 2025)

```
SELECT count(DISTINCT employee_number) AS hc
FROM dim_worker_snapshot
WHERE snapshot_date = DATE '2024-12-31'
AND worker_type = 'Regular'
AND employment_status IN ('Active', 'On Leave')
```

Try changing: replace '2024-12-31' with '2025-12-31'. Compare the two numbers. Now divide your Query 3 separations by each. How far apart are the two rates?

That spread is your Gotcha. Same numerator, different denominators, materially different rates. Write it up.

Compensation

Table: fct_compensation

Query 1 — Look at the raw data

```
SELECT * FROM fct_compensation LIMIT 5
```

Notice the columns. What currency are the salary figures in?

Here's what you'll see (using `.fetchdf()`):

	comp_id	worker_id	effective_date	base_salary_local	currency_code	target_bonus_pct	pay_grade
0	CMP0000001	W100000	2026-08-02	96900.0	USD	0.127	G5
1	CMP0000002	W100001	2025-04-15	140800.0	USD	0.169	G5
2	CMP0000003	W100002	2024-01-17	130500.0	USD	0.218	G8
3	CMP0000004	W100003	2025-10-23	190500.0	EUR	0.176	G8
4	CMP0000005	W100004	2023-06-08	126600.0	USD	0.145	G7

Reading the columns: `worker_id` = who this salary belongs to (joins to `dim_worker`). `effective_date` = when this salary took effect — dates span 2021-2026. `base_salary_local` = salary in *local* currency — check the next column. `currency_code` = USD (4,291), EUR (516), SGD (487). `target_bonus_pct` = bonus target as a decimal (0.127 = 12.7%).

Spot the trap: Row 3 is **EUR**, not USD. A naive `AVG(base_salary_local)` blends dollars, euros, and Singapore dollars into one meaningless number.

Query 2 — The naive average

```
SELECT round(avg(base_salary_local)) AS avg_salary
FROM fct_compensation
```

That's a number. Would you send it to your VP? What would you want to know first?

Query 3 — Break it out by currency

```
SELECT currency_code,
       count(*) AS n,
       round(avg(base_salary_local)) AS avg_salary
FROM fct_compensation
GROUP BY currency_code
ORDER BY 1
```

Compare the per-currency averages to the blended average from Query 2. How different are they? Does the blended number mean anything?

Query 4 — Who's included?

```
SELECT w.worker_type,
       count(*) AS n,
       round(avg(c.base_salary_local)) AS avg_salary
FROM fct_compensation c
JOIN dim_worker w ON w.worker_id = c.worker_id
```

```
WHERE c.currency_code = 'USD'

GROUP BY w.worker_type

ORDER BY 1
```

Try changing: remove the `WHERE currency_code = 'USD'` line. Does it matter?

Query 5 — Base salary vs total comp

```
SELECT round(avg(base_salary_local)) AS avg_base,
       round(avg(base_salary_local * (1 + target_bonus_pct))) AS avg_total
FROM fct_compensation
WHERE currency_code = 'USD'
```

"What do we pay people?" — does that mean base, or base plus bonus? How big is the gap?

Your Gotcha is in Query 3 or Query 5 — either the currency mixing or the base-vs-total ambiguity. Write up whichever one surprised you more.

Engagement

Table: fct_engagement_survey

Query 1 — Look at the raw data

```
SELECT * FROM fct_engagement_survey LIMIT 10
```

Notice the columns. How many items per person? What's the last column?

Here's what you'll see (using `.fetchdf()`, first 5 rows):

	response_id	wave_date	worker_id	item_code	response_value	scale_max
0	RS00000001	2023-04-15	W105183	ENG01	4	5
1	RS00000002	2023-04-15	W105183	ENG02	4	5
2	RS00000003	2023-04-15	W105183	ENG03	5	5
3	RS00000004	2023-04-15	W105183	MGR01	5	5
4	RS00000005	2023-04-15	W105183	MGR02	4	5

Reading the columns: `wave_date` = which survey wave (7 waves, 2023–2026). `item_code` = which question (ENG01–03, GRW01, MGR01–02) — each person has up to 6 rows per wave. `response_value` = their answer. **But look at `scale_max`.** `scale_max` = 5 (in 2023–24) or 7 (in 2025–26).

Spot the trap: A "4 out of 5" (80%) is not the same as a "4 out of 7" (57%). The scale changed mid-stream. Averaging raw `response_value` across years shows engagement "declining" even if sentiment didn't change.

Query 2 — The naive trend

```
SELECT year(wave_date) AS yr,
       round(avg(response_value), 2) AS avg_score
FROM fct_engagement_survey
WHERE item_code IN ('ENG01', 'ENG02', 'ENG03')
GROUP BY year(wave_date)
ORDER BY 1
```

Look at the year-over-year change. Is there a jump that looks unusually large? What could explain it?

Query 3 — Add one column and look again

```
SELECT year(wave_date) AS yr,
       scale_max,
       count(*) AS n,
       round(avg(response_value), 2) AS avg_score
FROM fct_engagement_survey
WHERE item_code IN ('ENG01', 'ENG02', 'ENG03')
GROUP BY year(wave_date), scale_max
ORDER BY 1
```

Compare this to Query 2. What changed between 2024 and 2025 besides the score?

Query 4 — Who responded?

```
SELECT w.worker_type, count(*) AS n  
FROM fct_engagement_survey e  
JOIN dim_worker w ON w.worker_id = e.worker_id  
GROUP BY w.worker_type  
ORDER BY 1
```

Your Define spec said Regular employees only. Can you filter that in the survey table without this join?

Your Gotcha is in Query 2 → Query 3. The jump that looked like improvement might be something else entirely. Write it up.

Shared Tables — What They Look Like

Every domain joins to these. Run `SELECT * FROM table_name LIMIT 5` to see them yourself.

dim_worker_snapshot — the main headcount table

97,705 rows · one row per worker per month-end · 42 monthly snapshots

	snapshot_date		worker_id	employee_number	worker_type	employment_status	org_unit_v2	department	location_id
	job_profile_id	hire_date	fte						
0	2026-05-31	W100000	E100000	Intern	Active	OU-2000	Sales	LOC007	
JP-04	2026-05-04	1.0							
1	2026-06-30	W100000	E100000	Intern	Active	OU-2000	NaN	LOC007	
JP-04	2026-05-04	1.0							
2	2025-01-31	W100001	E100001	Contingent	Active	OU-2200	NaN	LOC003	
JP-01	2025-01-15	1.0							
3	2025-02-28	W100001	E100001	Contingent	Active	OU-2200	Marketing	LOC003	
JP-01	2025-01-15	1.0							
4	2025-03-31	W100001	E100001	Contingent	Active	OU-2200	Marketing	LOC003	
JP-01	2025-01-15	1.0							

Three things to see right away: (1) Same person in rows 0 and 1 — without `WHERE snapshot_date =` you get 97,705 rows instead of ~4,300. (2) department has NaN values — only 54% populated, use `org_unit_v2` instead. (3) Rows 0-2 include an Intern and a Contingent worker — filter `worker_type = 'Regular'` for "employees."

dim_worker — current-state worker table

5,294 rows · one row per worker

worker_id	employee_number	worker_type	org_unit_v2	location_id	job_profile_id	hire_date	employment_status	department
0	W100000	E100000	Intern	OU-2000	LOC007	JP-04	2026-05-04	Active
NaN								
1	W100001	E100001	Contingent	OU-2200	LOC003	JP-01	2025-01-15	Active
Marketing								
2	W100002	E100002	Regular	OU-2200	LOC003	JP-07	2023-10-19	Active
Marketing								
3	W100003	E100003	Regular	OU-1100	LOC005	JP-16	2025-07-25	Terminated
Engineering								
4	W100004	E100004	Regular	OU-4000	LOC002	JP-02	2023-03-10	Active
NaN								

One row per worker, current state. Simpler than the snapshot, but no history. **Terminated workers are in here** (row 3) — this is not an "active workers" table.

dim_org_unit — org hierarchy lookup

15 rows · one row per org unit

org_unit_v2	org_unit_name	org_group	department
0	OU-1000	Engineering	Technology
			Engineering
1	OU-1100	Platform Engineering	Technology
			Engineering
2	OU-1200	Data & Analytics	Technology
			Engineering
3	OU-1300	Security Engineering	Technology
			NaN

4	OU-2000	Sales	Go-To-Market	Sales
---	---------	-------	--------------	-------

Join on org_unit_v2 to get names. **OU-1300, OU-3300, and OU-4200 have no department mapping** — three org units vanish if you group by department.

dim_job_profile — job title lookup

20 rows · one row per job profile

	job_profile_id	job_title	job_category	job_level
0	JP-01	Software Engineer I	Individual Contributor	1
1	JP-02	Software Engineer II	Individual Contributor	2
2	JP-03	Senior Software Engineer	Individual Contributor	3
3	JP-04	Staff Engineer	Individual Contributor	4
4	JP-05	Engineering Manager	People Manager	4

Join on job_profile_id for titles. job_category is Individual Contributor or People Manager.

dim_location — location lookup

7 rows · one row per location

	location_id	city	state_province	country_code	region
0	LOC001	Atlanta	GA	US	Americas
1	LOC002	Denver	CO	US	Americas
2	LOC003	Boston	MA	US	Americas
3	LOC004	Austin	TX	US	Americas
4	LOC005	Dublin	NaN	IE	EMEA

Join on location_id. Seven locations, three regions. Non-US locations have NaN for state_province and different currencies in comp.

Tables That Look Useful But Will Mislead You

These exist in the warehouse on purpose — the kind of table in every real data environment that's not wrong enough to delete and not right enough to trust.

rpt_headcount_daily — stale rollup

	as_of_date	org_unit_v2	headcount
0	2023-01-31	OU-1000	78
1	2023-01-31	OU-1100	58
2	2023-01-31	OU-1200	38
3	2023-01-31	OU-1300	3
4	2023-01-31	OU-2000	69

Last date: 2026-02-28 — four months stale. Returns total headcount of 4,168, which is plausible enough to look correct and wrong enough to matter. Use `dim_worker_snapshot` instead.

rpt_attrition_monthly — no denominator

	month	org_unit_v2	separations
0	2022-07-31	OU-1200	1
1	2022-08-31	OU-1100	1
2	2022-09-30	OU-4100	1
3	2023-02-28	OU-2200	1
4	2023-02-28	OU-4200	1

Three columns: month, org, separations. That's a numerator with no denominator. Computing a rate from this table requires silently choosing a denominator from elsewhere. Use `fct_separation` and get headcount from `dim_worker_snapshot` yourself.

vw_active_workers — too broad

	worker_id	employee_number	worker_type	org_unit_v2	location_id	employment_status
0	W100000	E100000	Intern	OU-2000	LOC007	Active
1	W100001	E100001	Contingent	OU-2200	LOC003	Active
2	W100002	E100002	Regular	OU-2200	LOC003	Active
3	W100004	E100004	Regular	OU-4000	LOC002	Active
4	W100005	E100005	Regular	OU-1200	LOC002	Active

"Active workers" includes interns and contractors (rows 0–1). Counting this table as "headcount" gives 4,368 instead of 3,647 — an overcount of 19.8%.

fct_job_record — fan-out trap

	job_record_id	worker_id	employee_number	org_unit_v2	job_profile_id	location_id	worker_type	effective_start
	effective_end	is_current	change_reason					
0	JR0000001	W100000	E100000	OU-2000	JP-04	LOC007	Intern	2026-05-04
9999	12-31	True	Hire					
1	JR0000002	W100001	E100001	OU-2200	JP-01	LOC003	Contingent	2025-01-15
2025	10-27	False	Hire					

2	JR0000003	W100001	E100001	OU-2200	JP-01	LOC003	Contingent	2025-10-28
9999-12-31		True	Manager Change					
3	JR0000004	W100002	E100002	OU-2200	JP-07	LOC003	Regular	2023-10-19
2024-07-03		False	Hire					
4	JR0000005	W100002	E100002	OU-2200	JP-07	LOC003	Regular	2024-07-04
2025-08-24		False	Manager Change					

12,282 rows for 5,294 workers — 2.32x inflation. Workers with transfers have multiple rows (W100001 in rows 1-2, W100002 in rows 3-4). Join this to something and count rows: your number is more than double the real answer, with no error message. Use `dim_worker_snapshot` for headcount.

Remember: change the SQL inside the triple quotes in `explore.py`, save, run. The `read_only=True` in the script means you can't break the database — experiment freely. Every query on this page has been verified against the workshop warehouse.